쿠버네티스 통합 모니터링 대시보드

애플리케이션부터 인프라까지 아우르는 'Full-Stack 관측성(Observability)' 대시보드 구축

- 애플리케이션 계층: 서비스의 트래픽, 성공률, P99 지연 시간 및 JVM 힙 메모리 상태를 모니터링하여 사용자 경험과 직결되는 지표를 관리한다.
- 컨테이너/파드 계층 (Pod): Pod의 리소스 사용량, 재시작 횟수, Probe 상태 및 CrashLoopBackOff와 같은 대기 원인을 실시간 추적하여 배포 안정성을 확보한다.
- 인프라/노드 계층 (Node): 개별 노드의 CPU/Memory/Disk 및 네트워크 트래픽을 감시하여 물리적 자원의 임계치 도달 및 OOM Kill 리스크를 사전에 방지한다.
- 로그 및 트레이싱 분석: 에러 스트림과 스팬(Span) 데이터를 연계하여 장애 발생 시 구체적인 에러 지점과 원인을 즉각 식별한다.

1. 애플리케이션 모니터링

- HTTP Requests Per Second: 현재 서비스에 들어오는 실시간 트래픽 양을 확인하여 시

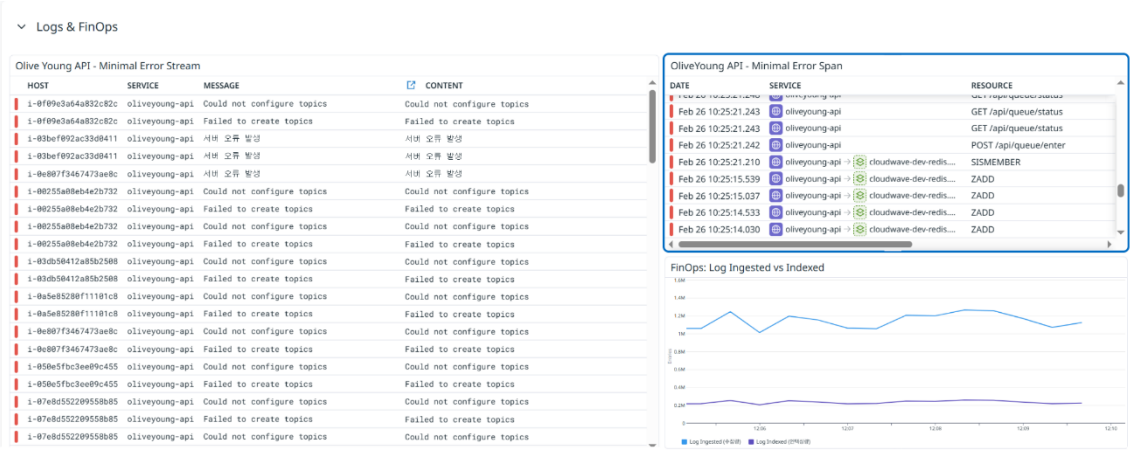
스템의 처리량을 가늠한다.

- Http Response Success Rate: 서비스가 얼마나 안정적으로 응답하고 있는지 보여주며, 에러 발생 시 즉각적인 대응 근거로 활용한다.

- Purchase API P99 Latency: 핵심 결제 API의 가장 느린 응답 시간을 확인하여 사용자 경험을 저해하는 로직 병목을 감시한다.

- JVM Heap Memory 사용률: 애플리케이션의 메모리 할당 상태를 모니터링하여, GC 부하로 인한 성능 저하나 메모리 누수에 의한 서버 다운(OOM) 가능성을 파악한다.

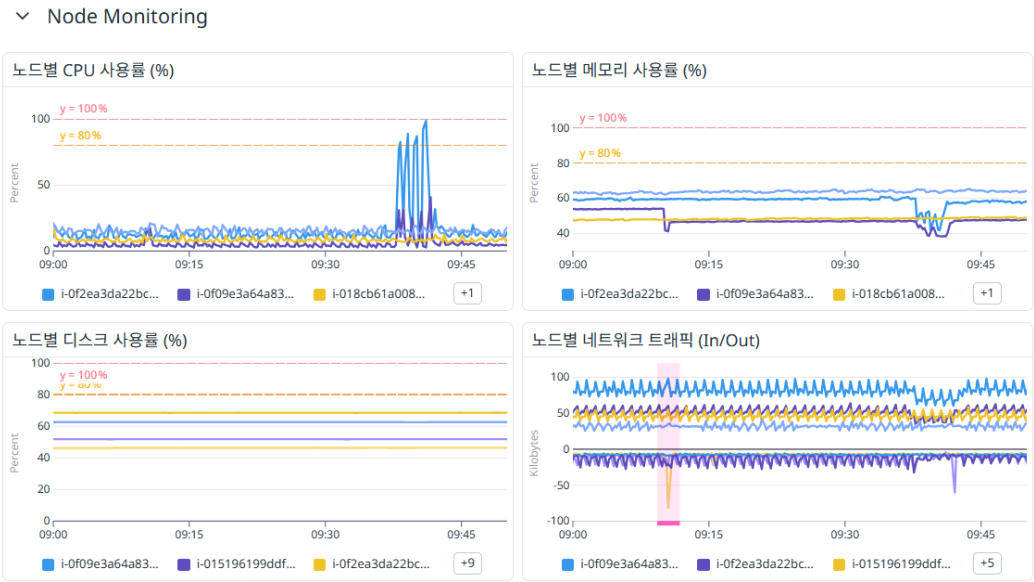
- Top Slowest Database Queries: 가장 느린 DB 쿼리를 식별하여 애플리케이션 지연의 주요 원인인 DB 병목 지점을 즉시 식별한다.



- Error Stream & Span: 애플리케이션에서 발생하는 에러 로그와 트레이스 정보를 실시간으로 나열한다. 특정 API 호출 시 어떤 에러가 발생했는지 즉시 확인하여 트러블슈팅 시간을 단축한다.

- Log Ingested vs Indexed: Datadog으로 유입된 전체 로그량(Ingested)과 실제 저장된 로그량(Indexed)을 비교한다. 불필요하게 많은 로그가 수집되어 비용이 과다 청구되는 것을 방지하기 위한 비용 관리 지표다.

2. 노드 모니터링



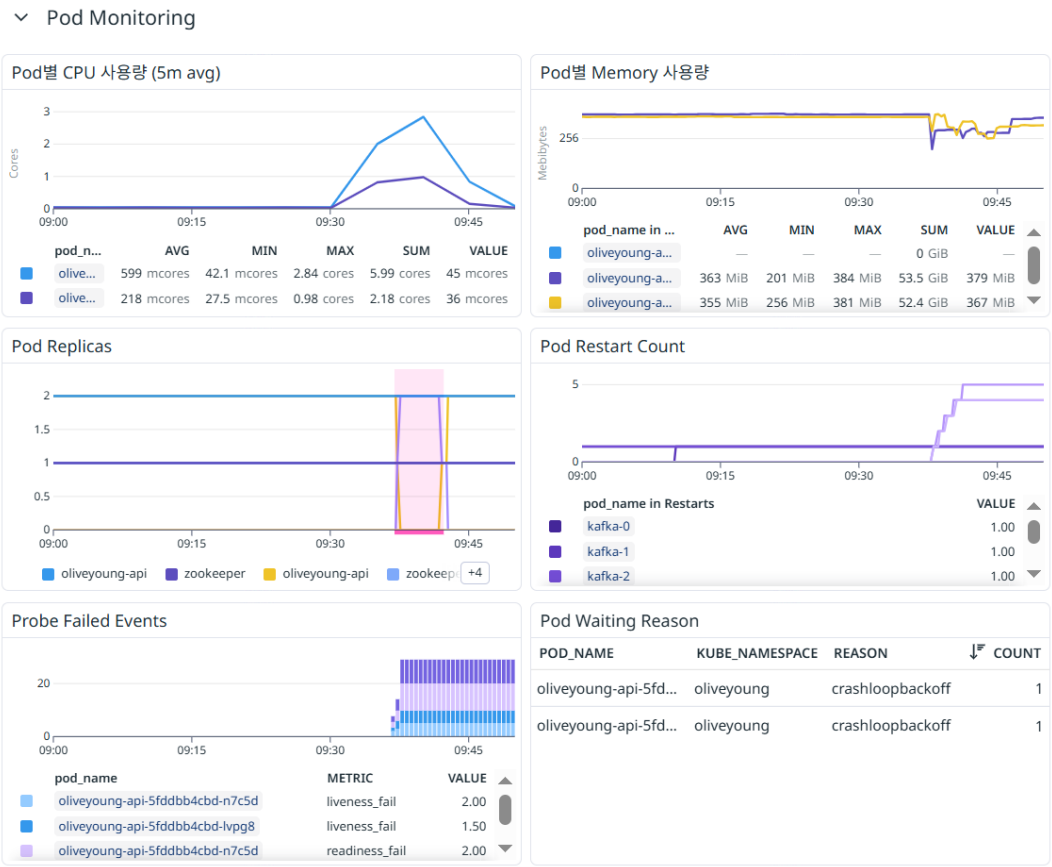
- 노드별 CPU 사용률: 특정 노드에 연산 부하가 집중되는지 확인한다. 그래프처럼 갑작스러운 스파이크가 발생할 경우, 어떤 프로세스가 자원을 독점하는지 파악하여 성능 저하를 방지한다.

- 노드별 메모리 사용률: 시스템 전체의 메모리 여유 공간을 감시한다. 80~100%에 도달하면 OS가 프로세스를 강제 종료(OOM Kill)할 수 있으므로, 적정 수준 유지 여부를 체크한다.

- 노드별 디스크 사용률: 로그 파일이나 임시 데이터로 인해 디스크가 가득 차는 상황을 방지한다. 디스크가 꽉 차면 더 이상 데이터를 쓸 수 없어 서비스 장애로 직결되기 때문에 상시 감시가 필요하다.

- 노드별 네트워크 트래픽: 데이터의 유입(In)과 유출(Out)량을 확인한다. 갑작스러운 트래픽 폭증이나 외부 공격(DDoS 등), 혹은 대량의 DB 쿼리로 인한 대역폭 병목을 식별하는 데 사용한다.

3. 파드 모니터링



- Pod별 CPU/Memory 사용량: 개별 컨테이너가 설정된 Resource Limit에 도달했는지 확인한다. 특정 Pod이 자원을 독점하거나 부족해지지 않도록 관리하여 성능 저하를 방지한다.

- Pod Replicas: 현재 실행 중인 Pod의 개수를 모니터링한다. 설정한 기대치(Desired)만큼 Pod이 정상적으로 떠 있는지 확인하여 서비스 가용성을 체크한다.

- Pod Restart Count: 애플리케이션의 비정상 종료 여부를 즉시 파악한다. 재시작 횟수가 잦다면 소스 코드의 버그나 메모리 부족(OOM) 등의 내부 결함을 의심할 수 있다.

- Probe Failed Events: Liveness/Readiness Probe 실패 여부를 감시한다. 서비스가 트래픽을 받을 준비가 되었는지, 혹은 좀비 상태가 되어 재시작이 필요한지 판단한다.

- Pod Waiting Reason: Pod이 실행되지 못하고 대기 중인 원인(CrashLoopBackOff, ImagePullBackOff 등)을 식별하여 배포 장애 시 빠른 트러블슈팅을 가능하게 한다.

성과 및 기대 효과

1. 장애 복구 시간(MTTR) 최소화: Probe Failed 이벤트와 Error Stream을 결합 분석하여

장애 인지부터 트러블슈팅까지 걸리는 시간을 획기적으로 단축했다.

2. 선제적 리소스 최적화: Top Slowest DB Queries와 노드 자원 지표를 통해 성능 병목 지점을 조기에 발견하고, 서비스 저하 전 튜닝 또는 확장을 결정하는 근거를 마련했다.

3. 모니터링 비용 최적화 (FinOps): Log Ingested vs Indexed 지표를 상시 모니터링하여, 불필요한 로그 수집으로 인한 과다 청구를 방지하고 모니터링 효율을 극대화했다.

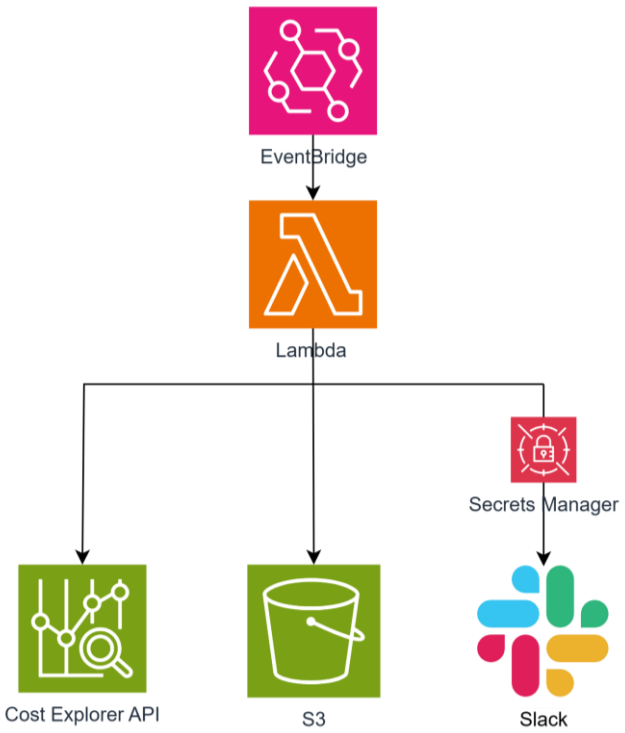
4. 가용성 보장: Pod Replicas와 Restart Count 감시를 통해 비정상적인 종료 상황에서도 설정된 기대 가용성을 유지할 수 있는 안정적인 운영 환경을 조성했다.

Serverless FinOps Pipeline

아키텍처 개요

클라우드 운영 중 발생할 수 있는 비정상적 비용 발생(인스턴스 미종료, 자원 과다 할당 등)을 방지하기 위해 구축한 기반 비용 모니터링 시스템이다. 매일 정기적인 리포트를 통해 클라우드 운영의 경제성과 가시성을 확보했다.

시스템 아키텍처



1. EventBridge (Daily Cron): 매일 오전 9시(KST) 정해진 스케줄에 따라 Lambda 함수를 실행하는 트리거 역할을 수행한다.

2. Lambda (Cost Collector): Cost Explorer API를 호출하여 서비스별 전일 비용 데이터를 수집 및 가공한다.

3. S3 (Archiving): 수집된 원본 JSON 데이터를 날짜별 경로에 아카이빙하여 향후 비용 분석을 위한 원천 데이터를 확보한다..

4. Secrets Manager (Security): Slack Webhook URL 등 민감 정보를 코드 내부가 아닌 별도 관리 서비스를 통해 호출함으로써 보안성을 강화했다.

5. Slack (Intelligent Delivery): 가공된 데이터를 바탕으로 요약본을 전송하며, 설정된 임계치를 기준으로 정상 범위일 때는 [REPORT] 요약본을, 초과 시에는 [ALERT] 요약본을 생성하여 메시지 가독성과 시각적 긴급도를 차별화했다.

성과 및 기대 효과

1. 비용 가시성 확보: 매일 오전 9시 전일 지출 내역을 팀원들과 공유함으로써 인프라 이상 징후를 조기에 발견하고 즉각 대응할 수 있는 안전한 시작 환경을 구축했다.

2. 사고 방지: 고사양 인스턴스 방치 등 실수로 인한 불필요한 과금 발생 시 즉각 대응이 가능한 모니터링 체계를 구축했다.

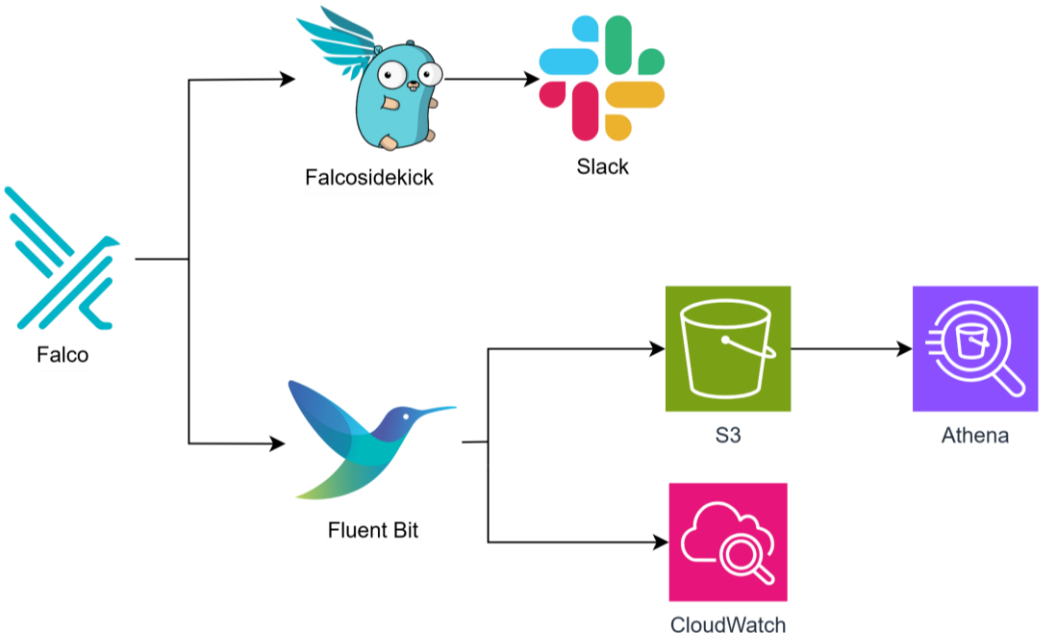
3. 보안성 강화: Secrets Manager 도입을 통해 외부 API 키 관리의 취약점을 제거하고 안전한 인프라 운영 환경을 조성했다.

EKS 보안 관측성 강화(Falco & Fluent Bit)

아키텍처 개요

런타임 보안 도구인 Falco가 생성하는 방대한 시스템 콜 로그를 효율적으로 관리하기 위한 통합 로그 파이프라인이다. 실시간 알림(Slack), 단기 고성능 분석(CloudWatch), 장기 저비용 보관(S3)의 3단계 계층 구조를 통해 운영 비용을 최적화하면서도 강력한 보안 감사 환경을 구축했다.

시스템 아키텍처



1. 실시간 알림 (Falcosidekick): 보안 규칙 위반 즉시 Slack으로 알림을 전송하여 초기 대응 속도를 극대화한다.

2. 로그 분류 및 태깅 (Fluent Bit): 로그 중 위험도가 높은 로그만 재분류하여 목적지별로 분기 처리합니다.

3. 지능형 분산 저장:

- CloudWatch: 선별된 위험 로그만 전송하며, 7일의 짧은 보존 기간을 통해 비용을 최소화하고 실시간 대시보드 성능에 집중한다.
- S3: 모든 원본 로그를 압축 보관하여 사고 후 분석(Forensics) 및 법적 증거 보존 용도로 활용한다.

성과 및 기대 효과

1. 비용 최적화: 모든 로그를 CloudWatch에 쌓지 않고 선별 수집 및 단기 보존(7일)을 적용하여 로그 관리 비용을 최소화했다.

2. 분석 효율성 증대: S3에 저장된 대용량 로그를 Amazon Athena와 연계하여 별도 서버 없이 SQL만으로 사후 분석(Forensics)이 가능하게 했다.

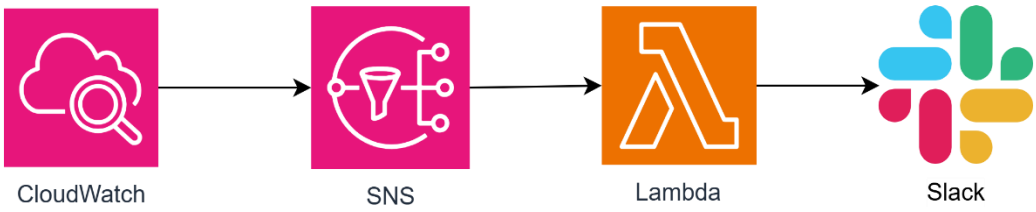
3. 안정적 리소스 관리: 경량 수집기인 Fluent Bit을 DaemonSet으로 배포하여, 보안 로그 수집이 실제 서비스 성능에 미치는 영향을 최소화했다.

RDS 실시간 장애 대응 및 모니터링 시스템

아키텍처 개요

데이터베이스의 가용성과 성능을 보장하기 위해 CloudWatch를 활용한 다각도 임계치 경보 시스템을 구축했다. 지표별로 정의된 임계치 도달 시, 즉각적인 알림 파이프라인을 통해 장애 발생 전 선제적인 대응이 가능한 환경을 마련했다.

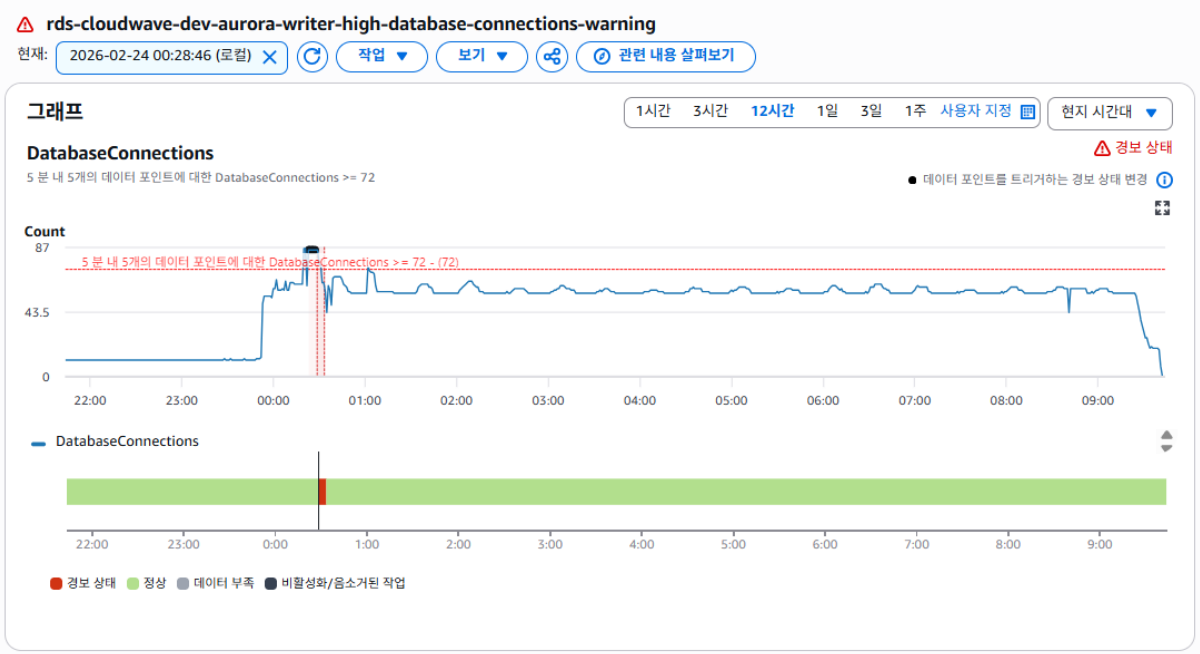
시스템 아키텍처



1. 탐지 (Detection): CloudWatch가 RDS의 주요 지표(CPU, Connection, Latency, Memory)를 5분 주기로 감시한다.

2. 전파 (Propagation): 임계치 초과 시 SNS(Simple Notification Service) 주제로 경보 메시지를 발행한다.

3. 가공 및 전송 (Processing & Delivery): Lambda 함수가 경보 데이터를 슬랙 포맷에 맞춰 가공하고, 웹후크(Webhook)를 통해 전담 채널에 실시간 알림을 전송한다.



성과 및 기대 효과

1. 장애 대응 시간(MTTR) 단축: 장애가 발생한 뒤 인지하는 것이 아니라, 임계치 도달 즉시 슬랙 알림을 수신함으로써 서비스 중단 전 빠른 트러블슈팅이 가능해졌다.

2. 데이터 기반의 인프라 최적화: Latency 및 Memory 지표를 통해 현재 RDS 사양의 적

정성을 판단하고, 불필요한 고사양 방지 또는 적기 스케일 업(Scale-up) 결정을 위한 객관적 근거를 확보했다.

3. 데이터베이스 가용성 극대화: 커넥션 및 메모리 고갈 문제를 사전에 차단함으로써, 부하가 몰리는 시점에도 안정적인 데이터베이스 연결 상태를 유지했다.